

CARRERA DE INGENIERÍA DE SOFTWARE

EXAMEN PRÁCTICO — UNIDAD 1

Generalidades de los Sistemas Distribuidos

Asignatura: Aplicaciones Distribuidas

Tipo: Examen práctico de unidad — Individual

Modalidad: Laboratorio con defensa técnica oral

Período académico: Regular 2025–2026 SPA

Código instrumento: EV-PR-U1

Docente: Ing. Guerrero Ulloa Gleiston Ciceron

Estudiante: Iván Andrés Villamarín Cuenca

Fecha: 06 de febrero de 2026

Repositorio: github.com/ivillamarinc/.../prueba

Objetivo

Implementar un prototipo funcional de un sistema distribuido que permita la comunicación y coordinación entre múltiples nodos, aplicando mecanismos básicos de sincronización, tolerancia a fallos, elección de coordinador y seguridad, con el fin de comprender el funcionamiento de los principales conceptos estudiados en la Unidad 1 de Aplicaciones Distribuidas.

Índice

Objetivo	1
1. Parte A — Comunicación entre procesos (20 %)	2
1.1. Cliente TCP — <code>tcp/ClienteTCP.java</code>	2
1.2. Servidor TCP — <code>tcp/ServidorTCP.java</code>	2
1.3. Delimitación de mensajes	4
1.4. Confirmación (ACK) al cliente	4
2. Parte B — Relojes lógicos de Lamport (20 %)	5
2.1. Reglas de Lamport	5
2.2. Clase Mensaje — marca temporal	5
2.3. Registro de resultados — <code>CsvLogger</code>	6
3. Parte C — Tolerancia a fallos (15 %)	6
3.1. Clase <code>HeartbeatManager</code>	7
4. Parte D — Coordinación — Algoritmo Bully (10 %)	7
4.1. Clase <code>BullyElection</code>	8
4.2. Ejecución del algoritmo Bully	8
4.3. Integración con Heartbeats	9
5. Parte E — Seguridad (10 %)	10
6. Ejecución en consola	10
6.1. Prueba de latencia TCP	11
7. Componente de análisis y defensa técnica	12
7.1. Propiedad CAP ante una interrupción de comunicación	12
7.2. Falacias de la computación distribuida	12
7.3. Tipos de transparencia	13
7.4. Propuesta de SLA de disponibilidad	13
7.5. Bully vs. consenso tipo Raft	13
8. Conclusión	13

1. Parte A — Comunicación entre procesos (20 %)

La comunicación entre procesos se implementó mediante **TCP** y **gRPC**. El cliente envía una operación al nodo servidor y recibe una confirmación de recepción, verificando que el intercambio de mensajes se realiza correctamente entre los componentes distribuidos.

1.1. Cliente TCP — tcp/ClienteTCP.java

```
1 package ec.edu.uteq.distribuidas.tcp;
2
3 import com.google.gson.Gson;
4 import ec.edu.uteq.distribuidas.common.Mensaje;
5 import ec.edu.uteq.distribuidas.common.Nodo;
6 import java.io.BufferedReader;
7 import java.io.InputStreamReader;
8 import java.io.PrintWriter;
9 import java.net.Socket;
10
11 public class ClienteTCP {
12     private static final String HOST = "localhost";
13     private static final Gson gson = new Gson();
14
15     public static Mensaje enviarMensaje(Nodo destino, Mensaje mensaje) {
16         try {
17             Socket socket = new Socket(HOST, destino.getPuertoTcp());
18             PrintWriter salida = new PrintWriter(
19                 socket.getOutputStream(), true);
20             BufferedReader entrada = new BufferedReader(
21                 new InputStreamReader(socket.getInputStream()))
22         } {
23             String json = gson.toJson(mensaje);
24             salida.println(json); // envio
25             String respuestaJson = entrada.readLine(); //
26             //repcion ACK
27             return gson.fromJson(respuestaJson, Mensaje.class);
28         } catch (Exception e) {
29             System.out.println("Error enviando mensaje TCP a "
30                 + destino.getNombre()
31                 + " por puerto "
32                 + destino.getPuertoTcp()
33                 + ": " + e.getMessage());
34             return null;
35         }
36     }
37 }
```

Listing 1: Implementación del cliente TCP

1.2. Servidor TCP — tcp/ServidorTCP.java

```
1 package ec.edu.uteq.distribuidas.tcp;
2
3 import com.google.gson.Gson;
4 import ec.edu.uteq.distribuidas.common.Mensaje;
5 import ec.edu.uteq.distribuidas.common.Nodo;
6 import ec.edu.uteq.distribuidas.common.RelojLamport;
7 import java.io.BufferedReader;
8 import java.io.InputStreamReader;
9 import java.io.PrintWriter;
10 import java.net.ServerSocket;
11 import java.net.Socket;
12
13 public class ServidorTCP {
14     private static final Gson gson = new Gson();
15     private final Nodo nodo;
16     private final RelojLamport reloj;
17
18     public ServidorTCP(Nodo nodo) {
19         this.nodo = nodo;
20         this.reloj = new RelojLamport();
21     }
22
23     public void iniciar() {
24         try (ServerSocket servidor = new ServerSocket(nodo.
25             getPuertoTcp())) {
26             System.out.println(nodo.getNombre()
27                 + " escuchando por TCP en puerto "
28                 + nodo.getPuertoTcp());
29             while (true) {
30                 Socket socket = servidor.accept();
31                 new Thread(() -> atenderCliente(socket)).start();
32             }
33         } catch (Exception e) {
34             System.out.println("Error en servidor TCP "
35                 + nodo.getNombre() + ": " + e.getMessage());
36         }
37     }
38
39     private void atenderCliente(Socket socket) {
40         try (
41             BufferedReader entrada = new BufferedReader(
42                 new InputStreamReader(socket.getInputStream()));
43             PrintWriter salida = new PrintWriter(
44                 socket.getOutputStream(), true)
45         ) {
46             String jsonRecibido = entrada.readLine();
47             Mensaje mensaje = gson.fromJson(jsonRecibido, Mensaje.
48                 class);
49             int relojActualizado = reloj.actualizar(mensaje.
50                 getTimestamp());
51
52             System.out.println "[" + nodo.getNombre() + "] Mensaje
53                 recibido");
54             System.out.println "Remitente: " + mensaje.getSender()
55                 ;
56             System.out.println "Timestamp recibido: " + mensaje.
57                 getTimestamp());
58         }
59     }
60 }
```

```

52         System.out.println("Reloj local actualizado: " +
relojActualizado);
53         System.out.println("Mensaje: " + mensaje.getMessage());
54         System.out.println("
-----");
55
56         Mensaje respuesta = new Mensaje(
57             nodo.getNombre(),
58             reloj.incrementar(),
59             "ACK recibido por " + nodo.getNombre()
60         );
61         salida.println(gson.toJson(respuesta));
62     } catch (Exception e) {
63         System.out.println("Error atendiendo cliente en "
64             + nodo.getNombre() + ": " + e.getMessage());
65     }
66 }
67 }

```

Listing 2: Implementación del servidor TCP

1.3. Delimitación de mensajes

Para evitar que varios mensajes enviados consecutivamente se mezclen o lleguen incompletos, se utilizó un mecanismo de delimitación basado en **líneas de texto**: cada mensaje JSON es enviado mediante `println()` y recibido mediante `readLine()`.

```

1  // Envío del mensaje (un JSON por línea)
2  String json = gson.toJson(mensaje);
3  salida.println(json);
4
5  // Recepcion del ACK
6  String respuestaJson = entrada.readLine();
7  return gson.fromJson(respuestaJson, Mensaje.class);

```

Listing 3: Framing por línea — envío y recepción

1.4. Confirmación (ACK) al cliente

Una vez procesado el mensaje, el nodo genera una respuesta de confirmación y la devuelve al cliente por la misma conexión TCP.

```

1  // Construcción del ACK
2  Mensaje respuesta = new Mensaje(
3      nodo.getNombre(),
4      reloj.incrementar(),
5      "ACK recibido por " + nodo.getNombre()
6  );
7  salida.println(gson.toJson(respuesta));
8
9  // Recepcion en el cliente
10 String respuestaJson = entrada.readLine();
11 return gson.fromJson(respuestaJson, Mensaje.class);

```

Listing 4: Generación y envío del ACK

2. Parte B — Relojes lógicos de Lamport (20 %)

2.1. Reglas de Lamport

Cada nodo mantiene un reloj lógico de Lamport con dos reglas fundamentales.

Incremento ante evento local:

```
1 public synchronized int incrementar() {
2     tiempo++;
3     return tiempo;
4 }
```

Listing 5: Incremento del reloj lógico local

Actualización al recibir un mensaje — se fija el reloj en $\max(\text{local}, \text{recibido}) + 1$:

```
1 public synchronized int actualizar(int recibido) {
2     tiempo = Math.max(tiempo, recibido) + 1;
3     return tiempo;
4 }
```

Listing 6: Actualización del reloj de Lamport al recibir un mensaje

2.2. Clase Mensaje — marca temporal

Cada mensaje intercambiado contiene una marca temporal lógica que permite sincronizar eventos distribuidos.

```
1 package ec.edu.uteq.distribuidas.common;
2
3 public class Mensaje {
4     private String sender;
5     private int timestamp;
6     private String message;
7
8     public Mensaje() { }
9
10    public Mensaje(String sender, int timestamp, String message) {
11        this.sender = sender;
12        this.timestamp = timestamp;
13        this.message = message;
14    }
15
16    public String getSender() { return sender; }
17    public int getTimestamp() { return timestamp; }
18    public String getMessage() { return message; }
19 }
```

Listing 7: Clase Mensaje con timestamp de Lamport**2.3. Registro de resultados — CsvLogger**

El sistema genera archivos CSV para almacenar los resultados obtenidos durante las pruebas de comunicación distribuida. El identificador de nodo actúa como criterio de desempate ante marcas temporales iguales.

```
1 package ec.edu.uteq.distribuidas.common;
2
3 import java.io.File;
4 import java.io.FileWriter;
5 import java.io.IOException;
6 import java.io.PrintWriter;
7
8 public class CsvLogger {
9     public static void guardarResultado(
10         String archivo, String metodo,
11         int envios, double latenciaPromedioMs) {
12
13         File file = new File(archivo);
14         boolean archivoNuevo = !file.exists();
15
16         try {
17             File carpeta = file.getParentFile();
18             if (carpeta != null && !carpeta.exists()) {
19                 carpeta.mkdirs();
20             }
21             try (PrintWriter writer = new PrintWriter(
22                 new FileWriter(file, true))) {
23                 if (archivoNuevo) {
24                     writer.println("metodo,envios,
25 latencia_promedio_ms");
26                 }
27                 writer.println(metodo + "," + envios
28                     + "," + latenciaPromedioMs);
29             }
30         } catch (IOException e) {
31             System.out.println("Error al guardar CSV: " + e.
32                 getMessage());
33         }
34     }
35 }
```

Listing 8: Logger de resultados en formato CSV**3. Parte C — Tolerancia a fallos (15 %)**

Los nodos intercambian latidos (*heartbeats*) periódicos. Si un nodo deja de responder, los demás lo marcan como caído dentro de un tiempo acotado, registran el evento y el sistema

continúa operando con los nodos restantes (gestión de fallo parcial).

3.1. Clase HeartbeatManager

La clase `HeartbeatManager` mantiene un registro de la última vez que cada nodo envió un latido, utilizando una estructura `ConcurrentHashMap` para acceso concurrente seguro entre múltiples hilos.

```
1 package ec.edu.uteq.distribuidas.common;
2
3 import java.util.Map;
4 import java.util.concurrent.ConcurrentHashMap;
5
6 public class HeartbeatManager {
7     private final Map<String, Long> ultimoLatido =
8         new ConcurrentHashMap<>();
9     private static final long TIMEOUT_MS = 5000;
10
11     public void registrarLatido(String nodo) {
12         ultimoLatido.put(nodo, System.currentTimeMillis());
13     }
14
15     public boolean estaActivo(String nodo) {
16         Long tiempo = ultimoLatido.get(nodo);
17         if (tiempo == null) {
18             return false;
19         }
20         return (System.currentTimeMillis() - tiempo) < TIMEOUT_MS;
21     }
22
23     public void verificarNodos() {
24         for (Nodo nodo : Nodo.TODOS) {
25             if (!estaActivo(nodo.getNombre())) {
26                 System.out.println(
27                     "[HEARTBEAT] "
28                     + nodo.getNombre()
29                     + " marcado como CAIDO "
30                 );
31             }
32         }
33     }
34 }
```

Listing 9: Gestión de heartbeats y detección de fallos

4. Parte D — Coordinación — Algoritmo Bully (10%)

Para garantizar la coordinación del sistema distribuido, se implementó el **algoritmo Bully**: el nodo activo con el ID más alto se convierte automáticamente en coordinador. Ante la caída del coordinador, el sistema dispara y completa una nueva elección sin intervención manual.

4.1. Clase BullyElection

```
1 package ec.edu.uteq.distribuidas.common;
2
3 public class BullyElection {
4     private Nodo coordinador;
5
6     public void iniciarEleccion() {
7         Nodo candidato = null;
8         for (Nodo nodo : Nodo.TODOS) {
9             if (nodo.isActivo()) {
10                 if (candidato == null
11                     || nodo.getId() > candidato.getId()) {
12                     candidato = nodo;
13                 }
14             }
15         }
16         coordinador = candidato;
17         if (coordinador != null) {
18             System.out.println(
19                 "[BULLY] Nuevo coordinador: "
20                 + coordinador.getNombre()
21             );
22         }
23     }
24
25     public Nodo getCoordinador() {
26         return coordinador;
27     }
28 }
```

Listing 10: Implementación del algoritmo Bully para elección de coordinador

4.2. Ejecución del algoritmo Bully

La figura 1 muestra la salida en consola del proceso de elección: cada proceso envía mensajes de elección a los de mayor ID, los nodos superiores responden con OK, y finalmente el proceso con el ID más alto es declarado coordinador y notifica al resto.

```

Introduzca el numero total de procesos
6
Proceso con el ID 5 falla
Proceso 0 envia mensaje de eleccion (0) al proceso 1
Proceso 0 envia mensaje de eleccion (0) al proceso 2
Proceso 0 envia mensaje de eleccion (0) al proceso 3
Proceso 0 envia mensaje de eleccion (0) al proceso 4
Proceso 1 envia mensaje de OK (1) al proceso 0
Proceso 2 envia mensaje de OK (2) al proceso 0
Proceso 3 envia mensaje de OK (3) al proceso 0
Proceso 4 envia mensaje de OK (4) al proceso 0
Proceso 1 envia mensaje de eleccion (1) al proceso 2
Proceso 1 envia mensaje de eleccion (1) al proceso 3
Proceso 1 envia mensaje de eleccion (1) al proceso 4
Proceso 2 envia mensaje de OK (2) al proceso 1
Proceso 3 envia mensaje de OK (3) al proceso 1
Proceso 4 envia mensaje de OK (4) al proceso 1
Proceso 2 envia mensaje de eleccion (2) al proceso 3
Proceso 2 envia mensaje de eleccion (2) al proceso 4
Proceso 3 envia mensaje de OK (3) al proceso 2
Proceso 4 envia mensaje de OK (4) al proceso 2
Proceso 3 envia mensaje de eleccion (3) al proceso 4
Proceso 4 envia mensaje de OK (4) al proceso 3
Finalmente el proceso 4 es el Coordinador
Proceso 4 envia mensaje de coordinador (4) al proceso 3
Proceso 4 envia mensaje de coordinador (4) al proceso 2
Proceso 4 envia mensaje de coordinador (4) al proceso 1
Proceso 4 envia mensaje de coordinador (4) al proceso 0
Fin de eleccion
BUILD SUCCESSFUL (total time: 7 seconds)

```

Figura 1: Salida de consola — ejecución del algoritmo Bully. El proceso 4 resulta coordinador y difunde el mensaje de coordinación al resto.

4.3. Integración con Heartbeats

La elección está integrada con el mecanismo de detección de fallos. Cuando un nodo deja de responder dentro del tiempo establecido por los heartbeats, se marca como caído y se dispara automáticamente una nueva elección:

```

1 if (!estaActivo(nodo.getNombre())) {
2     System.out.println(
3         "[HEARTBEAT] "
4         + nodo.getNombre()
5         + " marcado como CAIDO"
6     );
7     new BullyElection().iniciarEleccion();
8 }

```

Listing 11: Disparo automático de elección al detectar un fallo

5. Parte E — Seguridad (10 %)

Con el objetivo de impedir que clientes no autorizados ejecuten operaciones dentro del sistema distribuido, se implementó un mecanismo de **autenticación basado en tokens**. Antes de procesar cualquier solicitud, el nodo verifica que el cliente envíe un token válido; si la validación falla, la operación es rechazada y no se procesa.

```
1 package ec.edu.uteq.distribuidas.common;
2
3 public class Seguridad {
4     public static final String TOKEN_VALIDO = "TOKEN-UTEQ-2026";
5
6     public static boolean validar(String token) {
7         return TOKEN_VALIDO.equals(token);
8     }
9 }
```

Listing 12: Clase Seguridad — validación de token

6. Ejecución en consola

A continuación se presentan las capturas de la ejecución simultánea de los tres nodos del sistema distribuido, mostrando la recepción de mensajes con sus respectivos timestamps de Lamport.

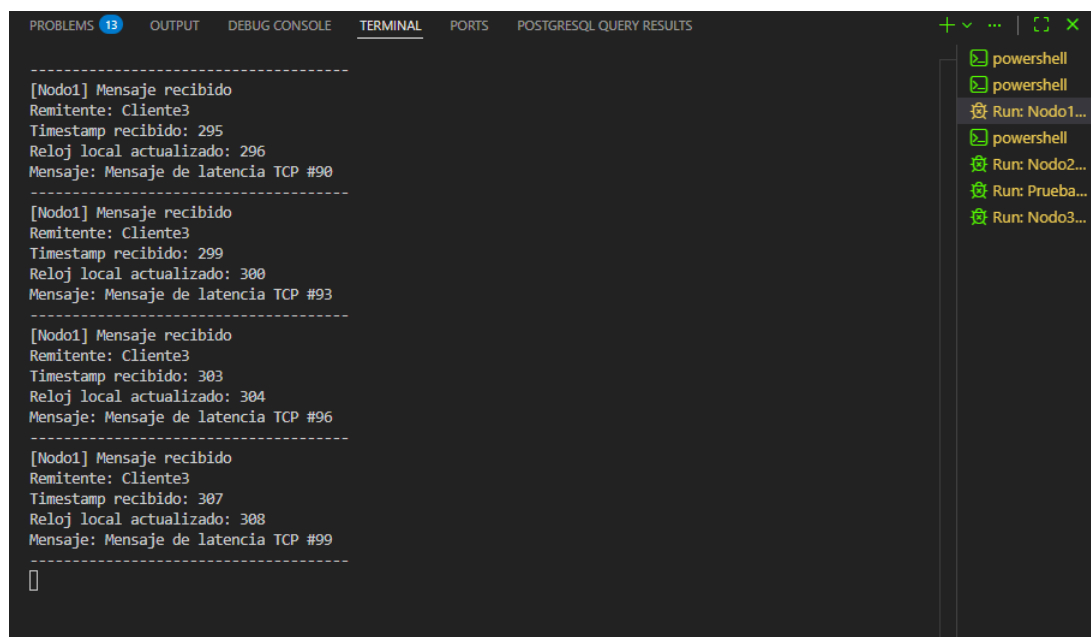
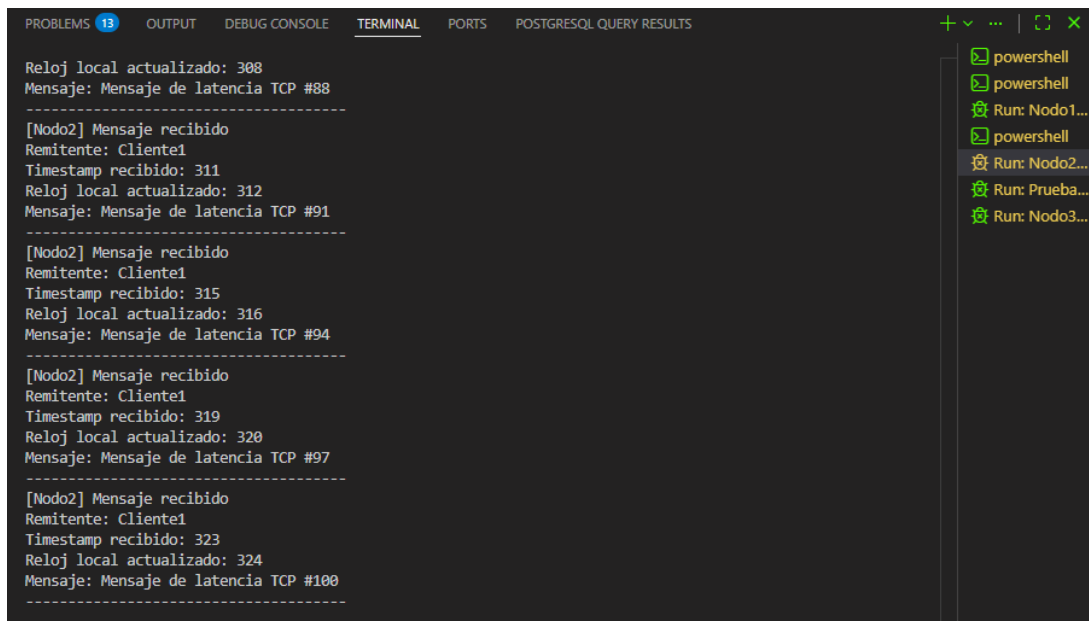
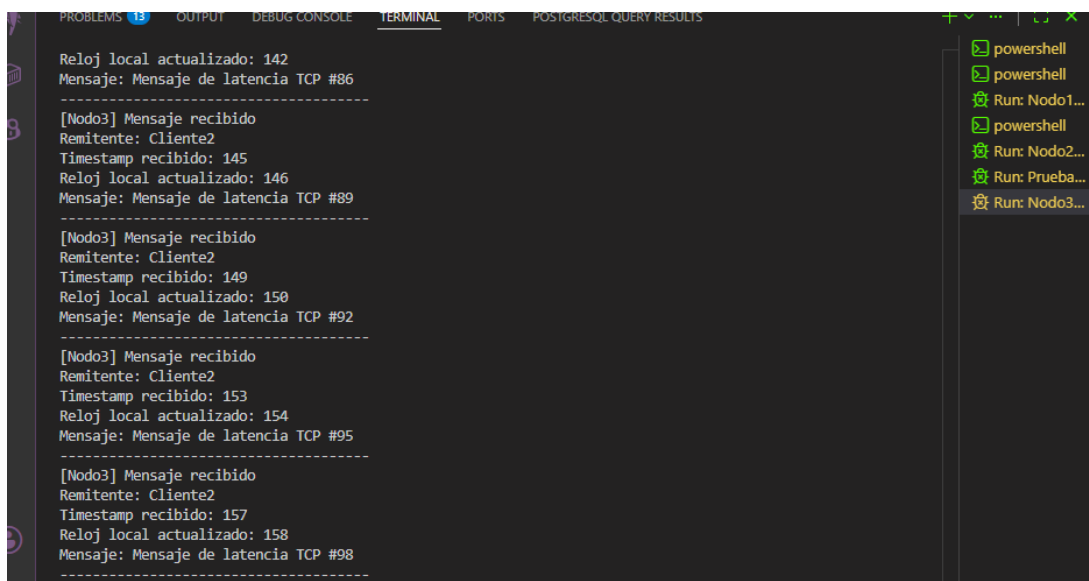


Figura 2: Consola de **Nodo 1** — recepción de mensajes desde Cliente3 con actualización del reloj lógico de Lamport.



```
PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRESQL QUERY RESULTS
Reloj local actualizado: 308
Mensaje: Mensaje de latencia TCP #88
-----
[Nodo2] Mensaje recibido
Remitente: Cliente1
Timestamp recibido: 311
Reloj local actualizado: 312
Mensaje: Mensaje de latencia TCP #91
-----
[Nodo2] Mensaje recibido
Remitente: Cliente1
Timestamp recibido: 315
Reloj local actualizado: 316
Mensaje: Mensaje de latencia TCP #94
-----
[Nodo2] Mensaje recibido
Remitente: Cliente1
Timestamp recibido: 319
Reloj local actualizado: 320
Mensaje: Mensaje de latencia TCP #97
-----
[Nodo2] Mensaje recibido
Remitente: Cliente1
Timestamp recibido: 323
Reloj local actualizado: 324
Mensaje: Mensaje de latencia TCP #100
-----
```

Figura 3: Consola de **Nodo 2** — recepción de mensajes desde Cliente1 con sincronización del reloj lógico.



```
PROBLEMS 13 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTGRESQL QUERY RESULTS
Reloj local actualizado: 142
Mensaje: Mensaje de latencia TCP #86
-----
[Nodo3] Mensaje recibido
Remitente: Cliente2
Timestamp recibido: 145
Reloj local actualizado: 146
Mensaje: Mensaje de latencia TCP #89
-----
[Nodo3] Mensaje recibido
Remitente: Cliente2
Timestamp recibido: 149
Reloj local actualizado: 150
Mensaje: Mensaje de latencia TCP #92
-----
[Nodo3] Mensaje recibido
Remitente: Cliente2
Timestamp recibido: 153
Reloj local actualizado: 154
Mensaje: Mensaje de latencia TCP #95
-----
[Nodo3] Mensaje recibido
Remitente: Cliente2
Timestamp recibido: 157
Reloj local actualizado: 158
Mensaje: Mensaje de latencia TCP #98
-----
```

Figura 4: Consola de **Nodo 3** — recepción de mensajes desde Cliente2 con timestamps crecientes.

6.1. Prueba de latencia TCP

La figura 5 muestra la ejecución de 20 intercambios TCP seguidos de la medición de latencia promedio sobre 100 envíos. El resultado — **1.104719 ms** de latencia promedio — se almacenó automáticamente en el archivo CSV `resultados_tcp.csv`.

```

--- Ejecutando 20 intercambios TCP ---
Intercambio #1 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 166
Intercambio #2 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 4
Intercambio #3 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 158
Intercambio #4 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 170
Intercambio #5 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 8
Intercambio #6 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 162
Intercambio #7 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 174
Intercambio #8 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 12
Intercambio #9 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 166
Intercambio #10 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 178
Intercambio #11 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 16
Intercambio #12 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 170
Intercambio #13 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 182
Intercambio #14 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 20
Intercambio #15 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 174
Intercambio #16 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 186
Intercambio #17 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 24
Intercambio #18 | Cliente3 envio a Nodo1 | respuesta: ACK recibido por Nodo1 | reloj cliente: 178
Intercambio #19 | Cliente1 envio a Nodo2 | respuesta: ACK recibido por Nodo2 | reloj cliente: 190
Intercambio #20 | Cliente2 envio a Nodo3 | respuesta: ACK recibido por Nodo3 | reloj cliente: 28

--- Midiendo latencia promedio de 100 envios TCP ---
Envios correctos: 100
Latencia promedio TCP: 1.104719 ms
Resultado guardado en src/main/resources/resultados/resultados_tcp.csv
PS C:\Users\Labo_0.02\Downloads\Prueba-comunicacion-distribuida\Prueba-comunicacion-distribuida>

```

Figura 5: Prueba de 20 intercambios TCP y medición de latencia promedio sobre 100 envíos. Latencia promedio: 1.104719 ms.

7. Componente de análisis y defensa técnica

7.1. Propiedad CAP ante una interrupción de comunicación

Cuando dos nodos pierden la comunicación entre sí, el sistema prioriza la **disponibilidad** (*Availability*). Esto significa que los nodos que siguen conectados continúan funcionando y procesando mensajes en lugar de detenerse por completo. Durante las pruebas realizadas se observó que, aunque un nodo dejara de responder, los demás podían seguir intercambiando información. Por esta razón, la solución busca mantener el servicio disponible aun cuando exista una falla de comunicación, sacrificando temporalmente la consistencia total entre nodos.

7.2. Falacias de la computación distribuida

Al desarrollar el sistema se tuvo en cuenta que la comunicación entre equipos no siempre funciona de manera perfecta. Los mensajes pueden tardar más de lo esperado o incluso no llegar a su destino. Por ello se configuraron tiempos de espera (`TIMEOUT_MS = 5000`) para detectar cuando un nodo deja de responder y evitar que el sistema quede esperando indefinidamente. También se consideró que la red puede cambiar y que los nodos pueden desconectarse en cualquier momento, lo que corresponde a las falacias de *“la red es confiable”*, *“la latencia es cero”* y *“la topología no cambia”* enunciadas por Deutsch y otros.

7.3. Tipos de transparencia

- **Transparencia de acceso:** Se ofrece parcialmente. Los usuarios pueden utilizar los servicios sin preocuparse por los detalles internos de la comunicación TCP o gRPC.
- **Transparencia de ubicación:** *No se ofrece completamente*, ya que es necesario conocer la dirección y el puerto de cada nodo de forma explícita.
- **Transparencia ante fallos:** Se ofrece de manera básica. El sistema permite que los nodos restantes continúen funcionando cuando uno presenta problemas; sin embargo, no recupera automáticamente el nodo afectado.
- **Transparencia de replicación:** *No se ofrece*, porque los datos no se copian ni sincronizan entre varios nodos. Cada nodo mantiene su propio estado de forma independiente.

7.4. Propuesta de SLA de disponibilidad

Se propone un nivel de disponibilidad del **99,9 %** anual. Considerando que un año tiene $365 \times 24 \times 60 = 525\,600$ minutos, el tiempo máximo de inactividad admisible se calcula como:

$$T_{\text{inactividad}} = 525\,600 \times (1 - 0,999) = 525\,600 \times 0,001 \approx \mathbf{525,6} \text{ minutos} \approx 8 \text{ h } 46 \text{ min}$$

Si el tiempo de falla acumulado en el año supera ese valor, no se estaría cumpliendo el acuerdo de disponibilidad establecido.

7.5. Bully vs. consenso tipo Raft

Reemplazar el algoritmo Bully por **Raft** aportaría una coordinación más confiable entre los nodos y una mejor capacidad para manejar fallos simultáneos, ya que Raft garantiza que solo un líder es elegido en cada término y que las decisiones se toman con quórum. Sin embargo, también introduciría un costo significativo: mayor complejidad de implementación, un número superior de mensajes intercambiados por ronda de elección y la necesidad de mantener un log de entradas replicadas para alcanzar consenso.

Para este prototipo, **Bully** resulta una opción adecuada por su sencillez, mientras que Raft suele emplearse en sistemas de producción donde la confiabilidad y la tolerancia a particiones son prioridad.

8. Conclusión

La práctica permitió comprender y aplicar los principios básicos de los sistemas distribuidos mediante la implementación de un prototipo funcional con múltiples nodos. Aunque se

logró establecer comunicación, coordinación, tolerancia a fallos y seguridad básica, también se evidenció que mantener la disponibilidad y consistencia del sistema ante fallos representa un desafío importante. Esto nos deja en definitiva que los sistemas distribuidos requieren mecanismos cada vez más robustos a medida que aumenta su complejidad, por lo que las soluciones implementadas constituyen un principio para abordar escenarios reales más exigentes.



Inicio (/)

[← Atrás \(/alu_documentos?action=resumencuestionario&id=OOPPPQQRRRSSSRTQOQL \)](#)

Revisión de intento

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Estudiante	VILLAMARIN CUENCA IVAN ANDRES
Comenzado el	Martes, 2 de Junio de 2026, 07:57
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 2 de Junio de 2026, 08:07
Tiempo empleado	0:10:00.276781
Calificación obtenida	11.50 / 20.00
Calificación final	5.75 / 10.00

PREGUNTA

1

Finalizado

1.00 / 1.00

Un sistema cifra todos los datos en tránsito con TLS, pero almacena las contraseñas en texto plano en la base de datos. Un atacante que compromete la base de datos obtiene todas las contraseñas. ¿Qué principio de seguridad se violó?

Seleccione una alternativa:

- ☐ a. El no repudio o denegación de servicios (DOS), ya que el atacante cambió las contraseñas actuales.
- ☐ b. La integridad de los datos en tránsito
- ☐ c. La seguridad en reposo; el cifrado en tránsito no protege los datos almacenados.

- ☒ d. La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados. ✓

La respuesta correcta es: La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.

PREGUNTA

2

Finalizado

1.00 / 1.00

Un usuario inicia sesión correctamente con su contraseña, pero el sistema le impide eliminar un archivo que pertenece a otro usuario. ¿Qué mecanismos de seguridad actúan en cada momento?.

Seleccione una alternativa:

- ☐ a. Cifrado al iniciar sesión y autenticación al denegar la eliminación.
- ☐ b. Autorización en ambos casos.
- ☒ c. Autenticación al iniciar sesión y autorización al denegar la eliminación. ✓
- ☐ d. Autenticación en ambos casos.

La respuesta correcta es: Autenticación al iniciar sesión y autorización al denegar la eliminación.

PREGUNTA

3

Finalizado

1.00 / 1.00

¿Cuál afirmación distingue correctamente un sistema distribuido de un sistema paralelo de memoria compartida?.

Seleccione una alternativa:

- ☐ a. El sistema distribuido no admite fallos parciales.
- ☐ b. El sistema distribuido siempre procesa más rápido.
- ☐ c. El sistema paralelo no puede tener múltiples procesadores.
- ☒ d. En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes. ✓

La respuesta correcta es: En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.

PREGUNTA

4

Finalizado

0.00 / 1.00

Un sistema replicado requiere un quórum de mayoría (más de la mitad de los nodos) para confirmar una operación y mantener la consistencia. Con 5 réplicas, ¿cuántos fallos por caída simultáneos puede tolerar sin perder la capacidad de formar quórum?

Seleccione una alternativa:

- ☒ a. 3, porque con 2 réplicas vivas todavía hay mayoría. ✖
- ☐ b. 4, porque basta con que una réplica siga viva.
- ☐ c. 0, porque cualquier caída rompe el quórum.
- ☐ d. 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas

La respuesta correcta es: 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas

PREGUNTA

5

Finalizado

1.00 / 1.00

En el protocolo de 2PC, ¿cuál es su principal debilidad y por qué se produce?

Seleccione una alternativa:

- ☒ a. Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final. ✔
- ☐ b. Que es no determinista en su resultado, en vista de que la comunicación entre las 2PC puede romperse.
- ☐ c. Que requiere relojes físicos sincronizados entre los participantes y que no garantiza la atomicidad de la transacción
- ☐ d. Que es desbloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" quedan libres del bloqueo indefinidamente esperando la decisión final.

La respuesta correcta es: Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final.

PREGUNTA

6

Finalizado

0.00 / 1.00

¿Por qué los algoritmos de consenso como Paxos y Raft exigen la participación de una mayoría (quórum) de nodos para tomar decisiones, en lugar de uno solo o de la totalidad?

Seleccione una alternativa:

- ☐ a. Porque la mayoría reduce el ancho de banda total consumido, además porque un solo nodo es siempre suficiente para alcanzar el consenso
- ☐ b. Porque la mayoría siempre resulta más rápida que un solo nodo.
- ☒ c. Porque la falta de exigencia universal (a todos los nodos) impediría progresar ante cualquier fallo, mientras que la mayoría garantiza que dos quórumes cualesquiera siempre se solapan en al menos un nodo, evitando decisiones contradictorias. ✕
- ☐ d. Porque exigir a todos los nodos impediría progresar ante cualquier fallo, mientras que la mayoría garantiza que dos quórumes cualesquiera siempre se solapan en al menos un nodo, evitando decisiones contradictorias.

La respuesta correcta es: Porque exigir a todos los nodos impediría progresar ante cualquier fallo, mientras que la mayoría garantiza que dos quórumes cualesquiera siempre se solapan en al menos un nodo, evitando decisiones contradictorias.

PREGUNTA

7

Finalizado

0.50 / 1.00

Relacione el escenario y tipo de transparencia evidenciada:

Relacione la opción correcta:

Un objeto se guarda y se recupera de una base de datos sin que el programador administre su almacenamiento en disco.

Transparencia De Migración ✕

Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban.

Transparencia De Concurrencia ✓

Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad.

Transparencia De Ubicación ✓

Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note.

Transparencia De Persistencia ✕

La respuesta correcta es: Una página sigue respondiendo con normalidad aunque un servidor del clúster se cayó, sin que el usuario lo note. → Transparencia de fallos , Un usuario abre un archivo sin enterarse de que físicamente reside en un servidor de otra ciudad. → Transparencia de ubicación , Un objeto se guarda y se

recupera de una base de datos sin que el programador administre su almacenamiento en disco. → Transparencia de persistencia , Cien personas editan el mismo documento a la vez y el sistema preserva el orden correcto de los cambios sin que ellas lo perciban. → Transparencia de concurrencia

PREGUNTA

8

Finalizado

1.00 / 1.00

Un servicio se traslada de un servidor a otro durante la madrugada, cuando no hay clientes conectados; al reanudar la operación, los clientes lo siguen invocando con el mismo nombre lógico sin percibir cambio alguno. ¿Qué transparencia se evidencia principalmente?

Seleccione una alternativa:

- ☒ a. Transparencia de migración. ✓
- ☐ b. Transparencia de ubicación.
- ☐ c. Transparencia de reubicación.
- ☐ d. Transparencia de acceso.

La respuesta correcta es: Transparencia de migración.

PREGUNTA

9

Finalizado

1.00 / 1.00

Dos centros de datos de un banco mantienen copias sincronizadas de las mismas cuentas. Una falla en el enlace de fibra deja a ambos centros sin comunicación entre sí, aunque cada uno sigue funcionando por separado y recibiendo clientes. Ante esta situación, el sistema decide bloquear todas las operaciones de retiro hasta que se restablezca el enlace, prefiriendo que ningún cliente pueda operar antes que arriesgarse a que la misma cuenta termine reflejando saldos distintos en cada centro. Según el teorema CAP, ¿qué par de propiedades prioriza esta decisión?

Seleccione una alternativa:

- ☐ a. Consistencia y disponibilidad (CA)
- ☒ b. Consistencia y tolerancia a particiones (CP) ✓
- ☐ c. Las tres propiedades simultáneamente
- ☐ d. Disponibilidad y tolerancia a particiones (AP).

La respuesta correcta es: Consistencia y tolerancia a particiones (CP)

PREGUNTA

10

Finalizado

1.00 / 1.00

Se compara un sistema centralizado con uno distribuido bajo la misma carga. ¿Cuál es una desventaja inherente al distribuido que el centralizado no presenta?

Seleccione una alternativa:

- ☐ a. La existencia de un único punto de fallo total del sistema.
- ☐ b. La imposibilidad de escalar horizontalmente.
- ☐ c. La ausencia total de concurrencia.
- ☒ d. La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes.



La respuesta correcta es: La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes.

PREGUNTA

11

Finalizado

1.00 / 1.00

Un desarrollador usa sockets TCP para enviar por separado los mensajes "HOLA" y "MUNDO", pero en el receptor a veces llegan juntos como "HOLAMUNDO" en una sola lectura. ¿Cuál es la explicación correcta?

Seleccione una alternativa:

- ☐ a. El sistema cambió automáticamente a UDP.
- ☒ b. TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente ✓
- ☐ c. Es un error físico de la tarjeta de red
- ☐ d. TCP perdió y luego retransmitió un mensaje.

La respuesta correcta es: TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente

PREGUNTA

12

Finalizado

Relaciones: Garantía o comportamiento observado y su modelo/concepto

Relacione la opción correcta:

El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado.

Tolerancia A Fallos Bizantinos



Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras.

"Lee Tus Propias Escrituras" (Read-Your-Writes)



Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato.

Reloj Lógico De Lamport



Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso.

Consistencia Eventual



La respuesta correcta es: Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras. → Consistencia eventual , Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato. → Consistencia fuerte (linealizable) , El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado. → Quórum de mayoría , Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso. → "Lee tus propias escrituras" (read-your-writes)

PREGUNTA

13

Finalizado

En el algoritmo Bully (del matón), un nodo con identificador 3 detecta la caída del coordinador y lanza una elección enviando mensajes a los nodos de identificador mayor (4, 5 y 6). Los nodos 5 y 6 están caídos, pero el nodo 4 responde. ¿Quién será el coordinador final?

Seleccione una alternativa:

- ☐ a. Ninguno, porque existen nodos caídos durante la elección
- ☒ b. El nodo 6, porque es el de mayor identificador entre los nodos vivos. **X**
- ☐ c. El nodo 4, porque es el de mayor identificador entre los nodos vivos.
- ☐ d. El nodo 3, porque fue quien inició la elección.

La respuesta correcta es: El nodo 4, porque es el de mayor identificador entre los nodos vivos.

PREGUNTA

14

Finalizado

0.00 / 1.00

Relacione: Situación de seguridad y propiedad que se protege

Relacione la opción correcta:

Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor.

Autorización



El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito.

Confidencialidad



Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido.

Integridad



Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol.

Confidencialidad



La respuesta correcta es: El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito. → Integridad , Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor. → Autenticación , Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido. → No repudio , Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol. → Autorización

PREGUNTA

15

Finalizado

1.00 / 1.00

En los relojes lógicos de Lamport se cumple que si el evento a ocurre antes que b ($a \rightarrow b$), entonces $C(a) < C(b)$. Sin embargo, observar que $C(x) < C(y)$ NO permite concluir que $x \rightarrow y$. ¿Por qué?

Seleccione una alternativa:

- ☒ a. Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad. ✓
- ☐ b. Porque $C(x) < C(y)$ siempre implica necesariamente $x \rightarrow y$, para la sincronización de los procesos.
- ☐ c. Porque los relojes de Lamport no son monótonos crecientes.
- ☐ d. Porque Lamport exige relojes físicos perfectamente sincronizados, es decir, que deben dar los mismos valores en los mismos tiempos.

La respuesta correcta es: Porque dos eventos concurrentes (sin relación causal) también pueden recibir marcas ordenadas, de modo que el orden total no implica causalidad.

PREGUNTA

16

Finalizado

1.00 / 1.00

¿Cuál enunciado distingue correctamente la "tolerancia a fallos" de la "prevención de fallos"?

Seleccione una alternativa:

- ☐ a. La tolerancia a fallos elimina por completo todos los fallos posibles, mientras que la prevención pone en alerta al administrador sobre posibles fallos.
- ☐ b. La prevención de fallos solo es aplicable al hardware, mientras que la tolerancia a fallos es tanto al software como al hardware.
- ☒ c. La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran. ✓
- ☐ d. Son términos sinónimos.

La respuesta correcta es: La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran.

PREGUNTA

17

Finalizado

0.00 / 1.00

Un servicio de distribución de archivos observa que, mientras más usuarios se conectan, más capacidad total (ancho de banda y almacenamiento) tiene disponible el sistema. ¿Qué modelo arquitectónico explica mejor esta propiedad?

Seleccione una alternativa:

- ☒ a. Modelo híbrido, porque depende siempre de un servidor central para todas las operaciones. ✗
- ☐ b. Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.
- ☐ c. Cliente-servidor de tres capas, porque separa la lógica de negocio.

- ☐ d. Cliente-servidor puro, porque el servidor central crece proporcionalmente con la demanda.

La respuesta correcta es: Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

PREGUNTA

18

Finalizado

1.00 / 1.00

Una aplicación en Windows escrita en C# y otra en Linux escrita en Java deben intercambiar mensajes sin que ninguna conozca el sistema operativo ni el lenguaje de la otra. ¿Qué propiedad del middleware lo hace posible y cuál es su costo principal?

Seleccione una alternativa:

- ☐ a. Garantiza que el sistema nunca fallará; el costo es el almacenamiento, que en nuestros días eso no es una limitación.
- ☐ b. Elimina por completo la latencia de red; el costo es el ancho de banda que se cree que es ilimitado por eso no se toma en cuenta su afectación.
- ☒ c. Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización. ✓
- ☐ d. Convierte el sistema distribuido en uno centralizado; el costo es la escalabilidad, dejando al sistema sin las posibilidades de crecer.

La respuesta correcta es: Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

PREGUNTA

19

Finalizado

0.00 / 1.00

Un arquitecto debe decidir entre consistencia fuerte (mediante consenso Raft) o consistencia eventual para un contador de "me gusta" en una red social con millones de operaciones por segundo. ¿Cuál es la decisión más razonable y por qué?

Seleccione una alternativa:

- ☒ a. Consistencia general, porque el dominio tolera imprecisiones generales y conviene priorizar disponibilidad y rendimiento bajo niveles de carga promedio, siendo aceptable que el valor converja después. ✗
- ☐ b. Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.
- ☐ c. Consistencia fuerte, porque un conteo exacto en tiempo real es crítico para la seguridad del sistema.

☐ d. Consistencia fuerte, porque el consenso Raft no introduce latencia adicional, siendo indiferente, porque ambos modelos rinden igual bajo carga elevada

La respuesta correcta es: Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.

PREGUNTA

20

Finalizado

0.00 / 1.00

Sockets TCP

Relacione la opción correcta:

Sockets UDP

Invocación De Métodos Sobre Objetos Remotos Con Paso De Objetos Serializados Mediante Stubs.



Invocación remota de métodos (RMI)

Sincronización De Relojes Físicos De Los Nodos Mediante Señal Satelital.



Middleware orientado a mensajes (colas)

Flujo De Bytes Fiable Y Ordenado Que No Preserva Los Límites Entre Mensajes.



La respuesta correcta es: Middleware orientado a mensajes (colas) → Invocación de métodos sobre objetos remotos con paso de objetos serializados mediante stubs. , Invocación remota de métodos (RMI) → Datagramas sin garantía de entrega ni orden, a cambio de menor latencia. , Sockets UDP → Flujo de bytes fiable y ordenado que no preserva los límites entre mensajes.

NAVEGACIÓN

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

Correcto ☐ Parcial ☐ Incorrecto

